

Appendix: Artefact Evaluation

Gabriele Bellini¹ and Roberto Metere²

¹University of Milan, Italy

²University of York, UK

Formal Verification of Credential Provisioning in ISO 15118 Park-and-Charge and Introducing Secure Ownership Transfer

1 Badge claims

We claim two EAPLS badges: (i) **Artefacts Available** and (ii) **Artefacts Functional**.

1.1 Available

The artefact is hosted as a public Git repository at

[https://github.com/gabriele-bellini/
iso15118-credential-provisioning-sot-verification](https://github.com/gabriele-bellini/iso15118-credential-provisioning-sot-verification).



The repository can be cloned and browsed anonymously, with no account, login, or identity tracking required from downloaders, in line with the artefact evaluation (AE) hosting requirement. For the camera-ready/AE package we additionally mint a permanent, versioned snapshot with a DOI on Zenodo (and rely on Software Heritage’s automatic archival of public GitHub repositories) so that the artefact remains retrievable independently of GitHub itself; the DOI badge/link will be added to the repository’s `README.md` once minted. Both the Docker image and the source archive described in Sec. 3 are attached to that release.

1.2 Functional

The submitted artefact is the collection of ProVerif 2.05 models and the `verify.sh` driver script that reproduce every formal-verification claim made in the paper. The artefact is documented in `README.md` (per-scenario tables, expected verdicts) and each run produces machine-checked, reproducible evidence (ProVerif’s own `RESULT` lines, persisted to `<name>.log` and to a cumulative `verification_results.txt`), which satisfies the “evidence of proper validation” requirement. Sec. 4 covers the functional outcomes below.



2 Functional outcomes

F1 — Buyer protection under ownership transfer (Park-and-Charge, no/TPM-backed rekeying).

The paper’s ownership-transfer analysis (PnC scenarios N1–N4, T1–T4) shows that TPM-backed key material together with rekeying and revocation restores contract-key secrecy and signature-forgery resistance after a vehicle changes owner, whereas the non-TPM baselines (N1, N3, N4) do not, and that low-entropy signed material re-opens a TPM signing-oracle attack (scenario T3) that no revision of the protocol can remove by rekeying alone. This outcome is reproduced by the `pnc_ownership_transfer_scenarios/` models (buyer-protection subset, filenames matching `[nt][t_]*.pv`).

F2 — Seller protection and billing legitimacy (Park-and-Charge, billing-contract revocation).

The paper shows billing is always correctly attributed to the legitimate operator of the EV regardless of ownership, while seller protection (i.e., prevention of billing to a party that no longer owns the vehicle)

requires an explicit billing-contract revocation step (scenarios NR, TR) and fails without it (NK, TK). This outcome is reproduced by the `pnc_ownership_transfer_scenarios/` models (seller-protection subset, filenames matching `[nr][rk]*.pv`).

F3 — ISO 15118-2 certificate installation protocol.

The paper reports that the standard's certificate-installation exchange between EV and eMSP achieves secrecy, correctness, and EV's authentication of the eMSP, but *not* the eMSP's authentication of the EV. This outcome is reproduced by the `iso_certificate_install_verification/` model.

F4 — Forward/backward privacy of credential provisioning under key leakage.

Achieving forward and backward secrecy is fundamental in the context of ownership transfer: the ISO protocol based on the (e1,s1) NIST cannot provide either, signifying severe privacy and security attacks; so we proposed a new protocol based on (e2,s2) NIST. The paper shows that a naïve instantiation of the (e2,s2) NIST key-exchange pattern (as used by TLS) cannot be applied to our context, where it is found vulnerable to attacks under several single/double key-leakage combinations, while our proposed variant proves secure. This outcome is reproduced by the `credential_provisioning/` models (`dh-[fb]*.pv` for the baseline, `dh-var-[fb]*.pv` for our variant).

F5 — Resistance to TPM-as-signing-oracle abuse during provisioning.

The paper shows that treating the on-board TPM as a bare signing oracle during (e2,s2)-style provisioning admits impersonation and compromise attacks that our variant construction removes, while both retain forward and backward privacy. This outcome is reproduced by the `credential_provisioning/` models (`*e2s2-tpm*.pv` for the baseline, `*e2s2-var-tpm*.pv` for our variant).

F6 — End-to-end rekeying/ownership-transfer protocol.

The paper's proposed four-party (Buyer, Seller, OEM, EV) rekeying protocol achieves secrecy of all long-term and session keys, mutual authentication and message ordering between every pair of honest roles, and resistance to scam/rogue-OEM/impersonation attacks, at the cost of two accepted limitations: three reachability properties are not met by design (the protocol purposefully aborts on those traces) and a benign buyer double-transaction is not prevented (though it causes no desynchronisation or damage). This outcome is reproduced by the `proposed_ownership_transfer_protocol/` model.

3 Quick start

3.1 Directory structure

```

iso15118-credential-provisioning-sot-verification/
├── docker/
│   └── Dockerfile ..... ProVerif 2.05 image
├── pnc_ownership_transfer_scenarios/
│   ├── nt*.pv ... t*.pv ..... buyer-protection scenarios (F1)
│   ├── nr*.pv ... r*.pv ..... seller-protection scenarios (F2)
│   └── *.pvl ..... Shared ProVerif libraries
├── iso_certificate_install_verification/
│   └── EV-Certificate-Install-Protocol.pv .. Unmodified protocol (F3)
├── credential_provisioning/
│   ├── dh-*.pv, dh-var-*.pv ..... Fwd/bwd privacy (F4)
│   ├── *e2s2-tpm*.pv, *e2s2-var-tpm*.pv .... TPM oracle (F5)
│   └── *.pvl ..... Shared ProVerif library
├── proposed_ownership_transfer_protocol/
│   └── ownership-transfer.pv ..... Our rekeying protocol (F6)
├── verify.sh ..... Driver: runs proverif, etc.
└── README.md ..... Summary and instructions

```

AE reviewers only need `verify.sh` and the four model directories above; all other files (`*.log`, `analysis-*/graph dumps`, `verification_results.txt`) are generated by the quick-start and evaluation steps below and may be safely ignored or deleted between runs.

3.2 Environment: Docker image with ProVerif 2.05

The artefact package ships a `docker/Dockerfile` that follows the official ProVerif installation instructions¹ and installs it on the `PATH`, so that reviewers do not need to build OCaml/ProVerif themselves. The Dockerfile through OPAM.

```
git clone https://github.com/gabriele-bellini/iso15118-credential-provisioning-sot-
verification.git

cd iso15118-credential-provisioning-sot-verification

docker build -t iso15118-sot-verification:2.05 -f docker/Dockerfile .

docker run --rm -it \
  -v "$(pwd)":/artefact -w /artefact \
  iso15118-sot-verification:2.05 bash
```

In our tests, the docker image uses 252MB of disk space.

3.3 Sanity check (kick-the-tires)

Inside the container, confirm the tool and the driver script are operational before running any of the functional-evaluation steps of Sec. 4:

```
proverif -help | head -1          # confirms ProVerif 2.05 is on PATH

cd pnc_ownership_transfer_scenarios
../verify.sh [nt][t_]*.pv        # single, fast scenario, all ticks expected
```

The expected line summary is:

```
NK: Seller protection = ✗ - Billing legitimated = ✓.
NR: Seller protection = ✓ - Billing legitimated = ✓.
TK: Seller protection = ✗ - Billing legitimated = ✓.
TR: Seller protection = ✓ - Billing legitimated = ✓.
```

(`verify.sh` itself renders these as ✓, ✗, — symbols on a terminal. They correspond respectively to the raw `true`, `false`, and `cannot be proved` strings are what is persisted to the log files and are what we reproduce verbatim in the tables of Sec. 4.)

If this scenario check passes, the environment is ready for the full functional evaluation.

4 Functional evaluation

Estimated total time: under an hour on a laptop-class machine (each individual `verify.sh` invocation below completes in seconds to low minutes); no long-running experiments are involved, so no push-button batch harness beyond `verify.sh` itself is required.

All commands below are run from the artefact root inside the Docker container started in Sec. 3.

¹<https://bblanche.gitlabpages.inria.fr/proverif/install.html>

F1 & F2 — Park-and-Charge (PnC) ownership-transfer scenarios

```
cd pnc_ownership_transfer_scenarios
../verify.sh [nr][rk]*.pv      # F2: seller protection / billing legitimacy
../verify.sh [nt][t]*.pv      # F1: buyer protection
cd ..
```

Confirm `verification_results.txt` (buyer-protection run) matches Table 1, and (seller-protection run) matches Table 2.

Scenario	Billing legitimated	Contract-key secrecy	Sig. forgery resistance
N1	×	×	×
N2	✓	✓	✓
N3	×	×	×
N4	×	×	×
T1	✓	✓	✓
T2	✓	✓	✓
T3	✓	✓	×
T4	✓	✓	✓

Table 1: F1: buyer-protection verdicts (links to the corresponding rows of the paper’s ownership-transfer table).

Scenario	Seller protection	Billing legitimated
NK	×	✓
NR	✓	✓
TK	×	✓
TR	✓	✓

Table 2: F2: seller-protection / billing-legitimacy verdicts.

F3 — ISO 15118-2 certificate installation protocol

```
cd iso_certificate_install_verification
../verify.sh *.pv
cd ..
```

Expected result:

```
EV-Certificate-Install-Protocol: Secrecy = ✓ - Correctness = ✓ - eMSP Authentication
of EV = ✗ - EV Authentication of eMSP = ✓.
```

This matches F3: the protocol meets secrecy, correctness, and EV’s authentication of the eMSP, but not the eMSP’s authentication of the EV, exactly as stated in the paper.

F4 — Forward/backward privacy under key leakage

```
cd credential_provisioning
../verify.sh dh-[fb]*.pv      # baseline (e2,s2) instantiation
../verify.sh dh-var-[fb]*.pv  # our variant
cd ..
```

The baseline run should show ✗/— (privacy fails or cannot be shown) for every double-leakage combination and for most single-leakage combinations, while the variant run should show ✓ (privacy holds) uniformly across all six leakage combinations tested, confirming F4.

F5 — TPM-as-signing-oracle resistance

```
cd credential_provisioning
../verify.sh *e2s2-tpm*.pv      # baseline
../verify.sh *e2s2-var-tpm*.pv  # our variant
cd ..
```

Expected results:

```
(e2,s2)-tpm-oracle: Impersonation Resistance = ✗ - Forward privacy = ✓ - Unilateral
                    compromise security = ✗ - Partial post-compromise security = ✓ - Mutual compromise
                    security = ✗ - Backward privacy = ✓.

(e2,s2)-variant-tpm-oracle: Impersonation Resistance = ✓ - Forward privacy = ✓ -
                             Unilateral compromise security = ✓ - Partial post-compromise security = ✓ - Mutual
                             compromise security = ✓ - Backward privacy = ✓.
```

This confirms F5: the naïve baseline fails impersonation resistance and both compromise-security properties that our variant restores, while both retain forward/backward privacy.

F6 — End-to-end rekeying protocol

```
cd proposed_ownership_transfer_protocol
../verify.sh *.pv
cd ..
```

Reviewers should confirm that every property in `verification_results.txt` evaluates to ✓ except the ones explicitly discussed in the paper. All evaluate to ✗ by design, because the protocol intentionally aborts rather than reach states corresponding to the end of protocols. Interestingly, one is *buyer double transaction prevention*: this fails because a benign, harmless double transaction is not blocked. This matches F6 and the corresponding discussion in the paper.